

ПОЛНЫЙ СПИСОК СИГНАЛОВ И СЛОТОВ (QML → C++)

ГДЕ И ЧТО ЗАГОТОВЛЕНО

1 TIMELINE.CPP (основной класс)

Файлы:

- `timeline.h`
- `timeline.cpp`

Q_INVOKABLE функции (вызываются из QML):

```
cpp

// Управление клипами
Q_INVOKABLE bool addClip(const QString& filepath, int trackIndex, double startTime);
Q_INVOKABLE bool removeClip(int index);
Q_INVOKABLE bool moveClip(int index, int newTrackIndex, double newStartTime);
Q_INVOKABLE bool splitClip(int index, double splitTime);
Q_INVOKABLE bool trimClip(int index, double newTrimStart, double newTrimEnd);
Q_INVOKABLE bool applyEffect(int index, const QString& effectName, double value);

// Проекты
Q_INVOKABLE bool saveProject(const QString& filepath);
Q_INVOKABLE bool loadProject(const QString& filepath);

// Рендеринг
Q_INVOKABLE bool renderToFile(const QString& outputPath);
```

Сигналы (испускаются в QML):

```
cpp

signals:
    void currentTimeChanged();
    void totalDurationChanged();
    void clipsChanged();
    void clipAdded(int index);
    void clipRemoved(int index);
    void clipModified(int index);
    void renderProgress(int percent);
    void renderFinished(bool success);
```

Q_PROPERTY (двусторонняя связь):

```
cpp

Q_PROPERTY(double currentTime READ currentTime WRITE setCurrentTime NOTIFY currentTimeChanged)
Q_PROPERTY(double totalDuration READ totalDuration NOTIFY totalDurationChanged)
Q_PROPERTY(int clipCount READ clipCount NOTIFY clipsChanged)
```

2 CLIPMANAGER.QML

Файл: `/components/ClipManager.qml`

Что нужно изменить:

БЫЛО (mock):

```
qml

function addClip(filepath, trackNumber, startTime, duration) {
    var clipId = clips.length
    var mockDuration = duration || (Math.random() * 50 + 10) // ФЕЙК!

    var clip = {
        id: clipId,
        filepath: filepath,
        duration: mockDuration, // ФЕЙК!
        //...
    }

    clips.push(clip)
    revision++
}
```

СТАЛО (C++):

```
qml
```

```

function addClip(filepath, trackNumber, startTime) {
    console.log("📞 Вызываю cppTimeline.addClip")

    // Вызов C++ - он сам получит длительность через FFmpeg
    var success = cppTimeline.addClip(filepath, trackNumber, startTime)

    if (!success) {
        console.log("❌ Не удалось добавить клип")
    }
    // UI обновится автоматически через signal clipsChanged!
}

// Слушаем сигналы от C++
Connections {
    target: cppTimeline

    function onClipsChanged() {
        console.log("📡 clipsChanged от C++")
        updateClipsFromCpp() // Синхронизировать clips с C++
    }

    function onClipAdded(index) {
        console.log("📡 clipAdded:", index)
    }

    function onClipRemoved(index) {
        console.log("📡 clipRemoved:", index)
        removeClipFromArray(index)
    }

    function onTotalDurationChanged() {
        console.log("📡 totalDurationChanged")
        // TODO: обновить playbackManager.duration
        playbackManager.duration = cppTimeline.totalDuration
    }
}

```

TODO комментарии в файле:

- Строка 34: `thumbnailPath: "", // TODO: FFmpeg will generate thumbnail`
- Строка 47-50: TODO комментарий про FFmpeg thumbnail

3 VIDEOCLIP.QML

Файл: `/components/VideoClip.qml`

Заготовки:

Строка 13-15:

```
qml

property string thumbnailPath: "" // Путь к превью кадру (от FFmpeg)

// ===== ПРЕВЬЮ КАДРА =====
// TODO: FFmpeg will generate thumbnail
// cppTimeline.generateThumbnail(clipId, 0.0) → signal thumbnailReady(clipId, path)
```

Что нужно добавить в `Timeline.h`:

```
c++

// Генерация превью
Q_INVOKABLE void generateThumbnail(int clipIndex, double time);

signals:
    void thumbnailReady(int clipIndex, const QString& thumbnailPath);
```

Как использовать в QML:

```
qml

Component.onCompleted: {
    // Запросить превью при создании клипа
    cppTimeline.generateThumbnail(clipId, 0.0)
}

Connections {
    target: cppTimeline
    function onThumbnailReady(clipIndex, path) {
        if (clipIndex === root.clipId) {
            root.thumbnailPath = path
        }
    }
}
```

4 AUDIOCLIP.QML

Файл: `/components/AudioClip.qml`

Заготовки:

Строка 35-38:

```
qml

// ===== WAVEFORM CANVAS =====
// TODO: FFmpeg will extract audio samples
// cppFFmpeg.extractWaveform(audioPath) → signal waveformReady(samples[])
Canvas {
    property var waveformData: generateWaveformData() // Mock data

    // TODO: Подключение к FFmpeg
    // Connections {
    //     target: cppFFmpeg
    //     function onWaveformReady(filepath, samples) {
    //         if (filepath === root.audioPath) {
    //             waveformData = samples
    //             requestPaint()
    //         }
    //     }
    // }
    // }
    // }
}
```

Что нужно добавить (отдельный класс FFmpegHelper):

```
cpp

// ffmpeghelper.h
class FFmpegHelper : public QObject {
    Q_OBJECT
public:
    Q_INVOKABLE void extractWaveform(const QString& filepath);
    Q_INVOKABLE void generateThumbnail(const QString& filepath, double time);

signals:
    void waveformReady(const QString& filepath, const QVector<double>& samples);
    void thumbnailReady(const QString& filepath, const QString& thumbnailPath);
};
```

5 MAIN.QML (горячие клавиши)

Файл: `/main.qml`

Заготовки Undo/Redo:

Строки 420-433:

```
qml

// Редактирование (подготовка под C++)
Shortcut {
    sequence: "Ctrl+Z"
    onActivated: {
        // TODO: cppTimeline.undo()
        console.log("Undo")
    }
}

Shortcut {
    sequence: "Ctrl+Y"
    onActivated: {
        // TODO: cppTimeline.redo()
        console.log("Redo")
    }
}
```

Что нужно добавить в Timeline:

```
cpp

// Undo/Redo
Q_INVOKABLE void undo();
Q_INVOKABLE void redo();

signals:
    void undoAvailable(bool available);
    void redoAvailable(bool available);
```

Заготовки Delete:

Строка 438-443:

```
qml
```

```
Shortcut {
  sequence: "Delete"
  onActivated: {
    // TODO: Удалить выбранный клип
    console.log("Delete selected clip")
  }
}
```

Заготовки Cut (разрезать):

Строка 447-452:

```
qml

Shortcut {
  sequence: "C"
  onActivated: {
    // TODO: Razor tool (разрезать)
    console.log("Cut at playhead")
  }
}
```

Заготовки Markers:

Строка 456-468:

```
qml

// Маркеры (подготовка)
Shortcut {
  sequence: "M"
  onActivated: {
    // TODO: Добавить маркер
    console.log("Add marker at", playbackManager.currentTime)
  }
}

Shortcut {
  sequence: "Shift+M"
  onActivated: {
    // TODO: Перейти к следующему маркеру
    console.log("Go to next marker")
  }
}
```

6 EXPORTDIALOG (в main.qml)

Файл: `/main.qml`

Заготовки:

Строка 279-283:

```
qml

onClicked: {
    if (exportDialog.isRendering) {
        // TODO: cppTimeline.cancelRender()
        console.log("Отмена рендеринга")
    }
}
```

Строка 298-302:

```
qml

onClicked: {
    // TODO: Выбор пути сохранения
    // FileDialog для выбора выходного файла
    // TODO: cppTimeline.renderToFile(outputPath, format, resolution, quality)
}
```

Connections для прогресса:

```
qml

// TODO: Подключение к C++
// Connections {
//     target: cppTimeline
//     function onRenderProgress(percent) {
//         exportDialog.renderProgress = percent
//     }
//     function onRenderFinished(success) {
//         exportDialog.isRendering = false
//         if (success) {
//             console.log("✅ Экспорт успешен!")
//         }
//     }
// }
// }
```

7 SAVE/LOAD PROJECT

Файл: `/main.qml`

Строка 218:

```
qml

FileDialog {
    id: saveProjectDialog
    onAccepted: {
        console.log("Сохранен проект:", selectedFile)
        projectManager.currentProjectPath = selectedFile
        // TODO: cppTimeline.saveProject(selectedFile)
    }
}
```

Что уже есть в `Timeline.cpp`:

```
cpp

Q_INVOKABLE bool saveProject(const QString& filepath);
Q_INVOKABLE bool loadProject(const QString& filepath);
```

Как использовать:

```
qml
```

```
FileDialog {
    id: saveProjectDialog
    onAccepted: {
        var success = cppTimeline.saveProject(selectedFile)
        if (success) {
            console.log("✅ Проект сохранён")
        }
    }
}

FileDialog {
    id: openProjectDialog
    onAccepted: {
        var success = cppTimeline.loadProject(selectedFile)
        if (success) {
            console.log("✅ Проект загружен")
            // UI обновится автоматически через clipsChanged
        }
    }
}
```

📊 ИТОГОВАЯ ТАБЛИЦА

Файл	Функция	Статус	Сигнал
ClipManager.qml	addClip()	✅ Готов	onClipsChanged
ClipManager.qml	removeClip()	✅ Готов	onClipRemoved
ClipManager.qml	moveClip()	✅ Готов	onClipModified
VideoClip.qml	thumbnailPath	⌚ TODO	onThumbnailReady
AudioClip.qml	waveformData	⌚ TODO	onWaveformReady
main.qml	Undo/Redo	⌚ TODO	-
main.qml	Save/Load	✅ Готов	-
main.qml	Export	✅ Готов	onRenderProgress
main.qml	Markers	⌚ TODO	-

ПРИОРИТЕТ РЕАЛИЗАЦИИ

1. КРИТИЧНО (для работы)

- ☒ addClip() - ГОТОВО!
- ☒ removeClip() - ГОТОВО!
- ☒ moveClip() - ГОТОВО!
- ☒ getClipSourceDuration() - ГОТОВО!

2. ВАЖНО (для диплома)

-  generateThumbnail() - превью кадров
-  extractWaveform() - аудио визуализация
- ☒ saveProject() - ГОТОВО!
- ☒ loadProject() - ГОТОВО!

3. ХОРОШО БЫ (для плюсов)

-  renderToFile() - экспорт видео
-  undo/redo - отмена действий
-  splitClip() - разрезать клипы
-  markers - маркеры на timeline

СЛЕДУЮЩИЕ ШАГИ

1. Скопируй файлы в проект:

- `backend/timeline.cpp` → твой проект
- `backend/main.cpp` → замени свой main.cpp

2. Добавь в CMakeLists.txt:

```
cmake
```

```
target_sources(VideoEditorProject PRIVATE
    timeline.cpp
    timeline.h
    timelineclip.h
)

# FFmpeg
find_package(PkgConfig REQUIRED)
pkg_check_modules(LIBAV REQUIRED IMPORTED_TARGET
    libavformat
    libavcodec
    libavutil
)
target_link_libraries(VideoEditorProject PRIVATE PkgConfig::LIBAV)
```

3. **Измени ClipManager.qml** - убери mock, вызывай `cppTimeline`
4. **Тестируй:** Drag video → должна появиться РЕАЛЬНАЯ длительность!

ВСЁ ГОТОВО! ПОНЯЛ? 🎬 ✨